

Parameter and width

Reusable RTL names its bus widths once and threads them through ports, locals, and instances

One knob, many places

- Declare parameter WIDTH equals eight
- No hand-editing every bracket
- For memory, DEPTH sets how many words you have; address width tracks clog-two of depth
- For an adder, operands stay WIDTH wide but the sum is often WIDTH plus one to hold carry
- Localparam is the same idea but fixed inside the module

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Parameters

INTERACTIVE TOOL

Parameter / width explorer

Change `#(.WIDTH(N))` (and friends) and watch port ranges, address widths from `$clog2(DEPTH)`, and the instantiation update live. Starter: an 8-bit data path.

Starter example: parameterized bus with WIDTH=8 → logic [7:0] data_in/out and `#(.WIDTH(8))`.

Challenge 0 / 22 cleared

Quiz: `#(.WIDTH)`: In bus_slice `#(.WIDTH(16))`, the override...

☐ sets the module's WIDTH parameter to 16 for that instance

☐ only renames the module

☐ forces DEPTH=16

☐ is ignored unless localparam

Show hint

Check

Next

Load challenge setup

Idle

Quiz: `#(.WIDTH)`

Quiz: `[WIDTH-1:0]`

Quiz: `$clog2(16)`

Quiz: `$clog2(3)`

Quiz: `$clog2(1)`

Quiz: sum width

Quiz: localparam

Quiz: default vs override

Set WIDTH=8

Set WIDTH=16

Set WIDTH=4

Mem DEPTH=16

Mem DEPTH=32

DATA_W=32

Adder WIDTH=8

Adder WIDTH=4

FIFO DEPTH=8

FIFO DEPTH=16

Quiz: named override

Quiz: shared WIDTH

Quiz: DEPTH vs addr

WIDTH=1 edge

Explorer

Load starter example

TEMPLATE	WIDTH
----------	-------

Parameter width lab starter

Real Verilog practice

```
wsl - module09-param-width/examples/navigation

# Track A - real Unix
# real Verilog session (Track A)

$ cat examples/param_width/param_width.v
// param_width.v - parameter drives port ranges
module bus_slice #(
    parameter WIDTH = 8
)(
    input  [WIDTH-1:0] data_in,
    output [WIDTH-1:0] data_out
);
    assign data_out = data_in;
endmodule

module add_wide #(
    parameter WIDTH = 4
)(
    input  [WIDTH-1:0] a,
    input  [WIDTH-1:0] b,
    output [WIDTH:0]   sum
);
    assign sum = {1'b0, a} + {1'b0, b};
endmodule

$ iverilog -t null examples/param_width/param_width.v
(syntax check passed)
```

Real Verilog — parameterized bus and adder

Pitfalls to watch

- Do not hard-code bracket seven colon zero everywhere when WIDTH might change
- Do not forget sum needs WIDTH plus one when adding two WIDTH-bit values
- Do not assume clog-two of depth is always at least one
- Do not confuse parameter overrides at instantiation with port wiring

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, set WIDTH to sixteen and state the MSB index and bracket range
- On real Verilog, add a parameter to a module and override it in one instance
- When you are ready